CSCI 5411 Advanced Cloud Architecting | Summer 2026

Term Project Requirements

Report Due: July 5, 2026 | One-on-One Meeting: scheduled by TA

1. Project Overview

You will design, architect, and implement a production-quality, cloud-native application on AWS or an equivalent public cloud platform. Your work must demonstrate mastery of modern system design principles and compliance with all six pillars of the AWS Well-Architected Framework.

You are required to read the official AWS Well-Architected Framework documentation (https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html), then independently design, analyze, and implement your application in accordance with its principles and best practices. You are required to address all the six framework pillars.

Note that AWS Academy Learner Lab imposes certain restrictions that may prevent you from fully implementing all security measures — particularly around AWS IAM. Where implementation is not feasible, you are expected to discuss in your report how those security measures would be applied in a production environment, and explain what constraints prevented you from implementing them in the lab.

You are expected to go beyond surface-level descriptions: provide in-depth trade-off analysis, well-reasoned justification of every major design decision, and a rigorous evaluation of the system's quality attributes.

2. Domain Selection

There are no restrictions on your application domain — you are free to choose any problem space that genuinely interests you. After selecting a domain, you will design and develop the application independently.

To help you get started, here are some example application ideas for your chosen domains:

- Distributed data pipeline, recommendation engine backend, real-time analytics platform, multi-tenant SaaS API, video processing service, healthcare data exchange, fintech payment gateway.

These are merely suggestions — the possibilities are endless. Choose a domain that genuinely excites you and best showcases your skills and creativity.

3. Coding and Cloud Service Requirements

Your cloud service selection must satisfy all the following criteria:

- Non-trivial scope — your application must be sufficiently robust to support all the functionalities required by your chosen use case. This is not a programming course; you are welcome to use AI-assisted coding tools. However, you must clearly state what proportion of your code was AI-generated. No marks will be deducted regardless of the percentage.

Note: You are not allowed to directly copy code from external sources such as public repositories, online tutorials, or open-source projects. All code submitted must be written or generated by you. You are expected to design the logic, implement the solution, and debug and refine the code independently. Using third-party libraries and frameworks is permitted, but the application code itself must be your own original work.

- Cloud-native — leverage at least **three** cloud services spanning the following categories: compute, networking, messaging, AI (e.g., SageMaker, Bedrock), application integration (e.g., API Gateway, Load Balancer), or monitoring.
- Persistent state — store and retrieve data using at least **two** different storage solutions (e.g., relational database, object storage, NoSQL database, or cache).
- Real workload — handle real HTTP or other protocol traffic, event-driven triggers, or a scheduled pipeline.
- You must demonstrate your full CI/CD pipeline of the implementation.
- Operational — the system must be fully deployable via Infrastructure-as-Code (IaC) with minimal manual intervention.

4. System Design Requirements

Your report and design must rigorously address all five dimensions below. You are expected to critically discuss trade-offs and alternatives throughout.

4.1 Functional Requirements Analysis

Clearly define what the system does from both a user and business perspective.

- List all the functionalities of the chosen application.
- List at least **four** user stories or use cases of the chosen application, organized by priority (must-have vs. nice-to-have).

4.2 Non-Functional Requirements Analysis

Quantify quality attributes with measurable targets. For each requirement, explain how it influenced your architectural decisions. You may make reasonable assumptions to support capacity estimation, resource planning, and scaling policies. Below are some example assumptions. You may have many more detailed assumptions for your use case.

- Scalability: expected peak requests per second (RPS), and projected growth over one year.
- Availability: e.g., 99.95% availability = approximately 4.38 hours of downtime per year.
- Latency: p95 and p99 response time.
- Durability and RPO: data retention policy and acceptable data loss window.
- Security and compliance: relevant standards or regulations (e.g., GDPR, HIPAA, PCI-DSS), if applicable.
- Maintainability and deployability: CI/CD strategy, monitoring approach, and distributed tracing.

4.3 Architecture Design & Diagram

Provide two complementary diagrams:

- High-Level Architecture Diagram: shows all major components and services, data flows, communication protocols, and system entry points.
- Data Flow Diagram or Sequence Diagram: illustrates at least one critical end-to-end user journey at the request/response level.

You may use any diagramming tool of your choice — such as draw.io, Lucidchart, or Miro. Regardless of the tool used, each diagram must be accompanied by a written narrative that walks the reader through the key components and explains how data or requests flow through the system.

4.4 Tech Stack Selection & Justification

For every major component or service, provide:

- The technology chosen and its role in the system. For each choice, provide a clear rationale referencing concrete factors such as performance requirements, scalability needs, ecosystem maturity, or cost. For example, explain why you selected Java over alternative languages, or why a NoSQL database better fits your data model and access patterns compared to a relational database.
- An honest discussion of the limitations or risks associated with your chosen stack.

4.5 Implementation

Deliver a working, deployable implementation that meets the minimum expectations below. You may include screenshots in your report to demonstrate your implementation. You will also be expected to demo your IaC setup, CI/CD pipeline, and running application during the one-on-one meeting.

- Source code hosted on a private GitHub or GitLab repository.
- Full IaC coverage: every cloud resource must be provisioned via AWS CDK, CloudFormation, Terraform, or an equivalent tool. No manually created resources are permitted in the demo environment.
- CI/CD pipeline: at minimum, automated testing and deployment to a staging or production environment triggered on merge to the main branch.
- A README file containing an architecture summary, prerequisite setup steps, and complete deployment instructions.

5. Deliverables

Submit the following by 11:59 PM on July 5:

- Final report in PDF format.

6. Evaluation

The project grade is composed of two parts:

Final Report — 70%

Assessed on completeness, technical depth, quality of justifications, clarity of diagrams, and evidence of compliance with all six pillars of the AWS Well-Architected Framework. See the rubric in Section 7.

One-on-One TA Meeting — 30%

A 20-minute meeting with the TA, scheduled around the report deadline, structured as follows:

- Minutes 0–10: Student presentation. Walk the TA through your application, architecture, and key design decisions using a live demo.
- Minutes 10–20: TA questions. Expect in-depth questions on AWS Well-Architected Framework compliance, scalability design, failure scenarios, and performance results. Be prepared to trace a request end-to-end through your system.

You are required to present your student ID at the start of the meeting and keep your face visible on camera for its entire duration. Failure to comply with either requirement will result in a grade of zero for the meeting component.

7. Grading Rubric

7.1 Final Report (70 points total)

Component	Excellent (Full marks)	Satisfactory	Insufficient (0 pts)
Functional Requirements (8 pts)	At least 4 prioritized user stories or use cases, all actors identified and scenarios explained.	1–7 pts: User stories present but actors, or scenarios unclear.	Requirements absent or too vague to evaluate.
Non-Functional Requirements (8 pts)	All NFRs stated with measurable targets; each is explicitly linked to an architectural decision; capacity assumptions documented.	1–7 pts: Most NFRs present but lack measurable targets, architectural linkage, or capacity reasoning.	NFRs absent or entirely unmeasured.
Architecture Diagrams (10 pts)	Both diagrams present, clearly labelled, and complete; each accompanied by a written narrative explaining components and data flow.	1–9 pts: One diagram missing or key components/flows absent; narrative incomplete or absent.	No diagrams provided, or diagrams are unreadable.
Tech Stack Justification (8 pts)	Every major component justified with concrete rationale referencing at least one alternative; limitations and risks honestly acknowledged.	1–7 pts: Most choices justified but some lack rationale, alternative comparison, or risk discussion.	No justification provided.
Implementation (12 pts)	Full IaC coverage, working CI/CD pipeline, clear README, and deployment evidence all present.	1–11 pts: Most requirements met but IaC is partial, CI/CD is missing, or deployment evidence is insufficient.	No working implementation, or repository is inaccessible.
AWS Well-Architected Framework (all 6 pillars) (24 pts)	All six pillars addressed with concrete decisions, specific AWS services or features cited, and at least one trade-off discussed per pillar.	1–23 pts: Most pillars addressed but some lack depth, concrete service-level evidence, or trade-off discussion.	Fewer than three pillars addressed, or compliance is not evident.

7.2 One-on-One Meeting (30 points total)

Component	Excellent (Full marks)	Satisfactory	Insufficient (0 pts)
Presentation Quality (8 pts)	Well-organized and clear; covers architecture and key decisions with a live demo or visual walkthrough.	1–7 pts: Understandable but disorganized, or key sections of the system are missing from the presentation.	Unable to present the system coherently.
Design Decision Defense (12 pts)	Confidently defends all major decisions; can propose and critically evaluate alternatives when challenged.	1–11 pts: Defends most decisions but uncertain about trade-offs or unable to suggest alternatives on the spot.	Cannot explain why design decisions were made.
Technical Depth (10 pts)	Demonstrates graduate-level mastery of cloud architecture, system design decisions, and the AWS Well-Architected Framework principles and best practices.	1–9 pts: Solid foundational understanding but limited depth on trade-offs, failure scenarios, or the AWS Well-Architected Framework compliance.	Cannot answer technical questions at the expected graduate level.